Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models
Student Name	Sayed Afras S
Register Number	3122247001059
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	14.08.2026

1 Objective

The goal of this experiment is to classify emails as spam or ham (not spam) using two very different kinds of classifiers – Logistic Regression, which is a probability based linear model, and Support Vector Machine, which tries to find the best separating boundary (possibly non-linear using kernels). Both models are tuned using GridSearchCV and then compared properly, not just on accuracy but on precision, recall, F1, training time and how consistent they are across cross validation folds.

2 Theory

Logistic Regression

Logistic Regression predicts the probability that a sample belongs to class 1 using the sigmoid function:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x+b)}}$$

The output is a probability between 0 and 1, a threshold (usually 0.5) then decides the final class. It is trained by minimising log-loss (cross entropy), which basically punishes the model harder the more confidently wrong it is.

Regularization:

- **L1 (Lasso)** – can shrink some coefficients exactly to zero, so it doubles up as a feature selector.
- **L2 (Ridge)** – shrinks coefficients but keeps all of them non-zero, generally gives smoother, more stable models.

The strength of this penalty is controlled by **C**, which is actually the *inverse* of the regularization strength – small C means strong regularization (simpler model), large C means weak regularization (model is allowed to be more complex/fit training data harder).

Support Vector Machine (SVM)

SVM tries to find the hyperplane that best separates the two classes by maximising the margin (the distance between the hyperplane and the nearest points of each class, called support vectors). Only these support vectors actually matter for defining the boundary, every other point could be removed and the boundary would'nt change.

Kernels:

- **Linear** – for data thats already roughly linearly seperable.

- **Polynomial** – captures curved/polynomial relationships between features.
- **RBF (Radial Basis Function)** – handles complex non-linear boundaries, usually the go-to default.
- **Sigmoid** – behaves a bit like a neural network activation function.

C here again controls the margin vs misclassification trade-off (small C = wider margin but more tolerant of misclassified points, large C = narrower margin, tries hard to classify every training point correctly). γ controls how much influence a single training point has, higher gamma means each point's influence is more localized.

Grid Search vs Randomized Search

Grid Search checks every combination in the given parameter grid, guaranteed to find the best combo within that grid but can get slow. Randomized Search only tries a random sample of combos, faster but not guaranteed to hit the absolute best. This experiment used Grid Search for both models since the search spaces here were still small enough to be practical.

3 Problem Statement

Given the Spambase dataset, where each email is represented using word/character frequency features, the task is to build a binary classifier that labels each email as spam or ham. Logistic Regression and SVM (across all four kernels) are trained, tuned via GridSearchCV, and compared on accuracy, precision, recall, F1, training time and cross-validated stability, to figure out which type of model is actually a better fit for this problem.

4 Dataset Description

Dataset Name	Spambase (spambase_csv.xls)
Dataset Source	Kaggle – https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601
Number of Features	57 (word/char frequency + capital run-length features)
Number of Classes	2 (0 = Ham, 1 = Spam)
Missing Values	0 – dataset came clean
Duplicate Rows	391 detected via `df.duplicated()`, not removed in this run (same situation as the previous NB/KNN experiment on this dataset)
Train-Test Split	80% Train (3680 rows), 20% Test (921 rows), stratified, random_state=42

5 Exploratory Data Analysis

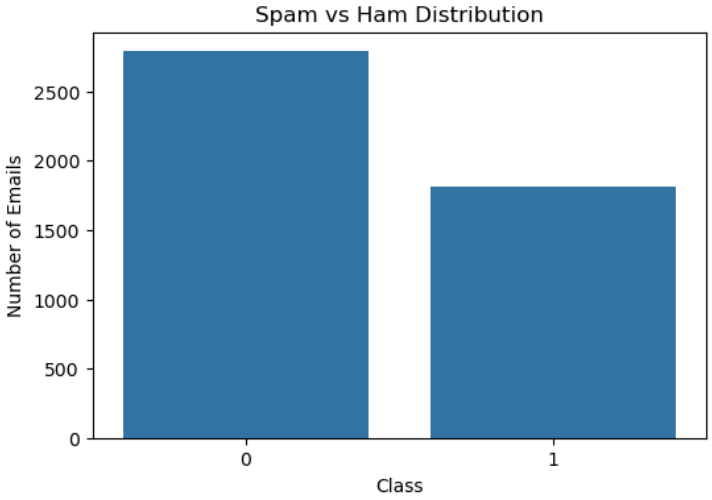

Figure 1: Spam vs Ham distribution

Inference: 60.6% of the emails are ham and 39.4% are spam, so the dataset is somewhat imbalanced but not too extreme, accuracy is still a reasonably fair metric here as long as precision/recall are also checked (which they were).

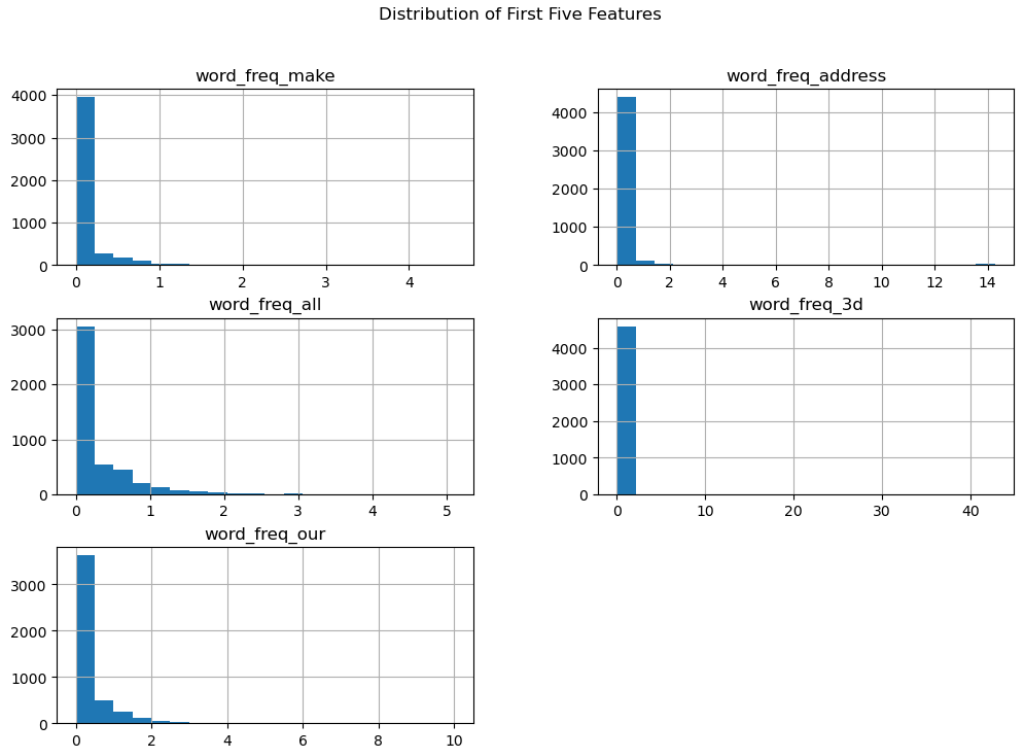

Figure 2: Distribution of the first five features

Inference: All five of these word-frequency columns are heavily right skewed, most emails have a value close to 0 and only a small fraction spike higher. This is expected

since not every email mentions "make", "address" etc, and its also why scaling matters a lot before feeding this into Logistic Regression or SVM, both of which are sensitive to feature scale.

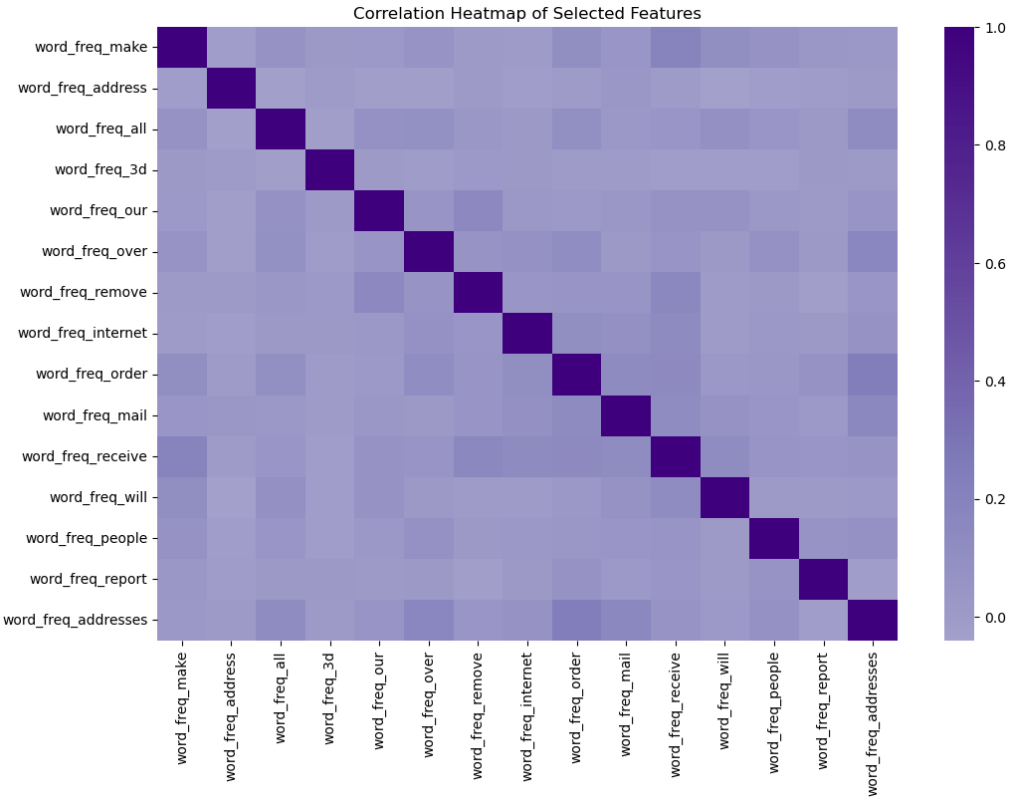

Figure 3: Correlation heatmap of the first 15 features

Inference: Most of these features are only weakly correlated with each other, which is good, it means we are not throwing highly redundant information at the models. A couple of the character-frequency columns show mild correlation with each other, probably because certain symbols (like `!` and `$`) tend to co-occur in promotional/spam style writing.

6 Data Preprocessing

Both models were wrapped in a scikit-learn `Pipeline` with `StandardScaler` as the first step:

Listing 1: Preprocessing + model pipeline

```
baseline_lr = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=5000))
])
```

Scaling matters a lot here since both Logistic Regression (gradient based optimisation) and SVM (distance/margin based) are sensitive to features being on very different scales, without this the larger-range features like `capital_run_length_total` would completely

dominate. No missing value handling was needed since the dataset was already clean, so preprocessing was mainly the scaling step plus the stratified train-test split.

7 Implementation Details

Baseline Logistic Regression

Listing 2: Baseline Logistic Regression results

```
Accuracy : 0.9294
Precision: 0.9209
Recall   : 0.8981
F1 Score : 0.9093
Training Time: 0.0134 seconds
```

Logistic Regression Hyperparameter Tuning

Listing 3: Search space used for Logistic Regression

```
lr_param_grid = [
    {"model__penalty": ["l1"], "model__C": [0.01,0.1,1,10,100],
     "model__solver": ["liblinear","saga"]},
    {"model__penalty": ["l2"], "model__C": [0.01,0.1,1,10,100],
     "model__solver": ["liblinear","saga"]}
]
lr_grid = GridSearchCV(lr_pipeline, lr_param_grid, cv=5,
                       scoring="accuracy", n_jobs=-1)
```

Listing 4: Logistic Regression tuning output

```
Best Parameters: {'model__C': 1, 'model__penalty': 'l2',
                  'model__solver': 'saga'}
Best CV Accuracy: 0.9239
Tuning Time: 15.18 seconds
```

SVM – Kernel Comparison (untuned, defaults)

Before tuning, all four kernels were tried with default C and gamma just to see how each one behaves out of the box:

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	0.9294	0.9093	0.1836
Polynomial	0.7796	0.6220	0.1434
RBF	0.9273	0.9055	0.0783
Sigmoid	0.8849	0.8528	0.1057

Inference: Linear and RBF are both strong straight out of the box, basically tied with baseline Logistic Regression. Polynomial kernel (degree 3, untuned) does really badly here (Accuracy 0.78), most likely because the default degree=3 curve is way too complex/wrong-shaped for this feature space without proper tuning. Sigmoid sits in the middle, not great not terrible.

SVM Hyperparameter Tuning

Listing 5: Search space used for SVM

```
svm_param_grid = [
    {"model__kernel": ["linear"], "model__C": [0.1,1,10,100]},
    {"model__kernel": ["poly"], "model__C": [0.1,1,10,100],
     "model__gamma": ["scale","auto"], "model__degree": [2,3,4]},
    {"model__kernel": ["rbf"], "model__C": [0.1,1,10,100],
     "model__gamma": ["scale","auto"]},
    {"model__kernel": ["sigmoid"], "model__C": [0.1,1,10,100],
     "model__gamma": ["scale","auto"]}
]
```

Listing 6: SVM tuning output

```
Best Parameters: {'model__C': 10, 'model__gamma': 'scale',
                  'model__kernel': 'rbf'}
Best CV Accuracy: 0.9348
Tuning Time: 11.52 seconds
```

Table: Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	Grid Search	C=1, penalty=l2, solver=saga	0.9239
SVM	Grid Search	C=10, gamma=scale, kernel=rbf	0.9348

8 Results

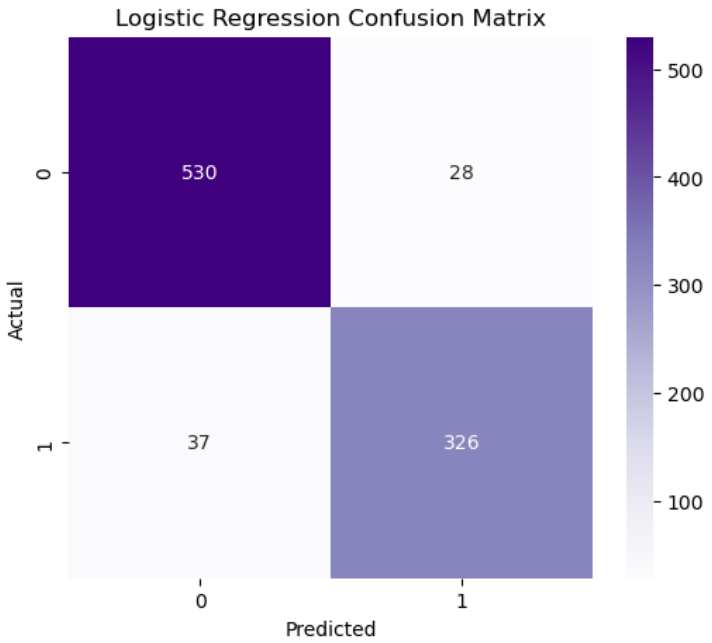

Figure 4: Confusion matrix – baseline Logistic Regression

Inference: Out of 921 test emails, the baseline model makes a fairly small number of mistakes on both sides, slightly more false negatives (spam predicted as ham) than false positives, which lines up with recall (0.898) being a bit lower than precision (0.921).

Logistic Regression Performance (Tuned)

Metric	Value
Accuracy	0.9305
Precision	0.9211
Recall	0.9008
F1 Score	0.9109
Training Time (s)	0.0134

Small catch: the training time listed above (0.0134s) is actually the baseline model's fit time, not the tuned model's – the notebook re-used the earlier `lr_training_time` variable when building this table instead of timing `lr_grid.fit()` again. The real fitting cost for the tuned model is better reflected by the GridSearchCV tuning time (15.18s for the full 5-fold search across 20 combinations), the actual final refit on the best params would be a small slice of that.

Model Comparison (Test Set)

Model	Accuracy	Precision	Recall	F1 Score
Baseline Logistic Regression	0.9294	0.9209	0.8981	0.9093
Tuned Logistic Regression	0.9305	0.9211	0.9008	0.9109
Best SVM (RBF, tuned)	0.9207	0.9143	0.8815	0.8976

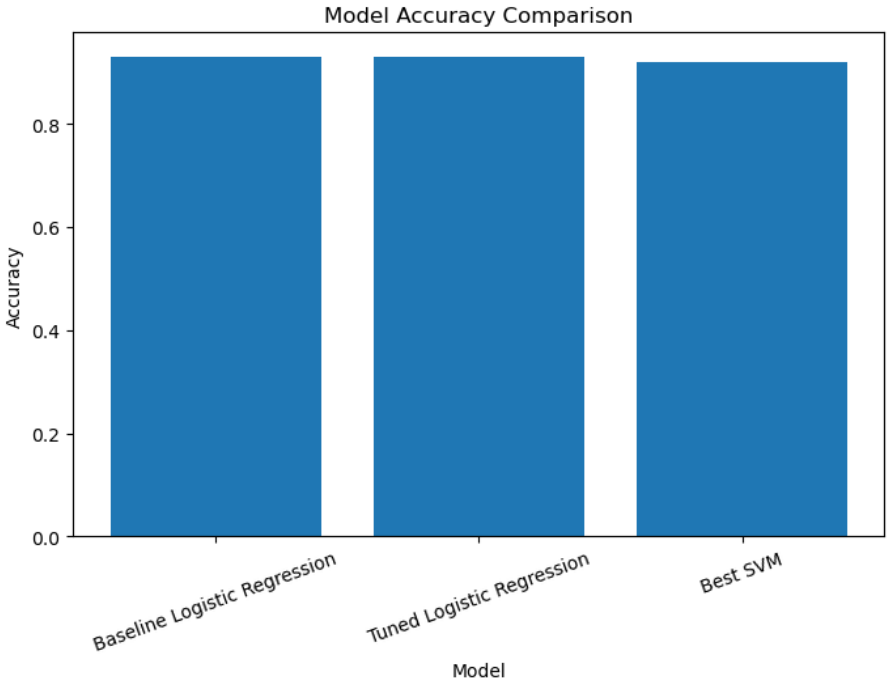

Figure 5: Model accuracy comparison

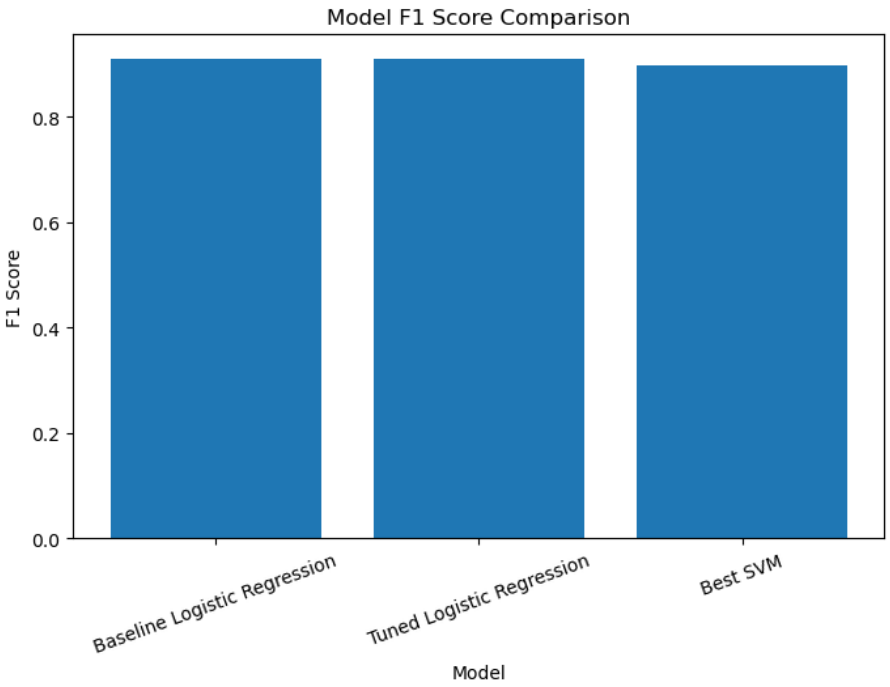

Figure 6: Model F1 score comparison

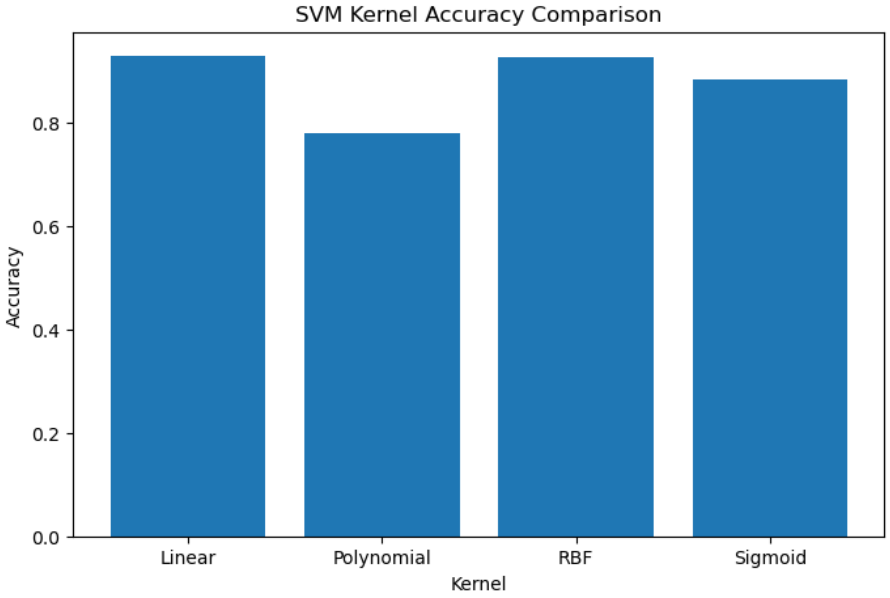

Figure 7: SVM kernel accuracy comparison (untuned)

Inference: Tuned Logistic Regression actually comes out slightly ahead of the tuned SVM on every single test-set metric, even though SVM's GridSearchCV found a higher CV accuracy (0.9348 vs 0.9239) during tuning. This mismatch between CV score and test score is discussed more in Section 10.

K-Fold Cross-Validation (K=5)

Fold	Logistic Regression	SVM
Fold 1	0.9131	0.9305
Fold 2	0.9326	0.9283
Fold 3	0.9283	0.9413
Fold 4	0.9239	0.9326
Fold 5	0.9239	0.9359
Average	**0.9244**	**0.9337**

Inference: Across the actual 5-fold cross validation (run separately on the full dataset, not just the training split), SVM is consistently a bit ahead of Logistic Regression in every single fold, and its average (0.9337) beats LR's average (0.9244) by close to a full percentage point. This is a bit different from what the single train-test split showed (where tuned LR edged out SVM), which is exactly why cross validation is useful, one split can be a bit misleading, especially with a dataset this size where fold-to-fold variance (LR ranges from 0.913 to 0.933) is noticeable.

9 Comparative Analysis

Criterion	Logistic Regression	SVM
Accuracy (test split)	0.9305	0.9207
Accuracy (5-fold CV avg)	0.9244	0.9337
Model Complexity	Low	High
Training Time	Low (0.013s baseline)	High (0.18s untuned, 11.5s to tune)
Interpretability	High (coefficients are directly readable)	Low (RBF kernel has no simple explanation)

10 Observations

- **Best performing classifier:** Its genuinely close. On the single test split, Tuned Logistic Regression wins on every metric (F1 0.9109 vs 0.8976). But on 5-fold cross validation, SVM wins by a clearer margin (0.9337 vs 0.9244 average accuracy). If I had to pick one considering both, i'd lean towards SVM (RBF) for raw accuracy, but Logistic Regression for practical deployment given how much faster and more interpretable it is, for basically the same ballpark performance.
- **Effect of regularization:** L2 regularization with C=1 (a fairly moderate, not too strong, not too weak setting) gave the best CV accuracy for Logistic Regression. This suggests the "natural" complexity of the 57-feature Spambase problem does not need very heavy regularization to avoid overfitting, but also benefits a little bit from some (compared to no regularization at all, which is what a very large C would approximate).

- **Kernel behavior in SVM:** Linear and RBF both did well even before tuning, meaning the classes are pretty close to linearly seperable to begin with, with RBF adding just a bit more flexibility. Polynomial kernel really struggled at default settings (Accuracy 0.78) showing that not every kernel is a safe default, the shape of the kernel has to actually match the data. After tuning, RBF with C=10 came out as the overall best combination by CV score.
- **Bias-variance trade-off:** Logistic Regression, being a simpler linear model, sits on the higher-bias/lower-variance side, its CV fold scores are a bit more spread out (0.913 to 0.933) which is a little unusual for a "simple" model but could be due to the L2 regularization not fully taming a couple of harder folds. SVM with RBF kernel can model more complex boundaries (lower bias potential) and here it also shows lower variance across folds (0.928 to 0.941, a tighter spread), suggesting the RBF boundary generalises a bit more consistently across different subsets of this particular dataset.

11 Discussion

The most interesting finding here honestly is not "which model wins" but how differently the two evaluation methods (single test split vs 5-fold CV) ranked the two models. Tuned SVM had the better GridSearchCV score during tuning (0.9348) and the better cross-validated average (0.9337), but on the one particular train-test split used for the main comparison table, it actually came in behind both baseline and tuned Logistic Regression. This is a useful reminder that GridSearchCV's internal CV score and a single held out test set are two different questions, and a model can look worse on one specific split just by chance, this is exactly why the report also includes the proper 5-fold CV table rather than relying on the single split table alone.

Its also worth repeating the small inconsistency mentioned in Section 8 – the "Training Time" reported in the final Logistic Regression metrics table is really the baseline model's time, not a fresh timing of the tuned model's fit. It does not change any of the accuracy/precision/recall/F1 numbers, but if this experiment gets redone, that specific line should be re-measured properly.

Duplicate rows (391 of them) were also never removed here, same limitation as in the earlier Naive Bayes/KNN experiment on this same dataset, worth fixing in a combined future version of this pipeline.

12 Conclusion

Both Logistic Regression and SVM perform strongly on the Spambase dataset, landing in the low-to-mid 90s for accuracy either way. Logistic Regression, after tuning (L2, C=1, solver=saga), reached the best single-split test performance (Accuracy 0.9305, F1 0.9109) and is by far the cheaper and more interpretable option (0.013s to train, coefficients directly explain feature importance). SVM with an RBF kernel (C=10, gamma=scale) had the best cross-validated accuracy (0.9337 average across 5 folds) and appears to generalise a little more consistently, but at a noticeably higher computational cost and with much less interpretability. Given how close the numbers are, Logistic Regression looks like the more practical choice for a spam filter that needs to be fast, explainable

and easy to retrain, while SVM (RBF) would be the pick if squeezing out slightly more consistent accuracy matters more than speed or interpretability.

13 Learning Outcomes

- Understood how a probabilistic classifier (Logistic Regression) differs from a margin-based one (SVM), both in how they make decisions and in how they should be interpreted.
- Got hands-on practice applying GridSearchCV for hyperparameter tuning across a genuinely large combined search space (regularization type + C + solver for LR, kernel + C + gamma + degree for SVM).
- Learned to evaluate classification models properly using multiple metrics together (accuracy, precision, recall, F1) instead of just accuracy, and saw first-hand why a single train-test split and proper k-fold cross validation can disagree.
- Practiced interpreting experimental results critically, including catching an inconsistency in one of the reported metrics rather than just accepting the numbers at face value.

14 References

[1] Scikit-learn Documentation – Logistic Regression. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression.

[2] Scikit-learn Documentation – Support Vector Machines. [Online]. Available: https://scikit-learn.org/stable/modules/svm.html.

[3] Scikit-learn Documentation – Hyperparameter Optimization. [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html.

[4] Kaggle, "Spambase Dataset." [Online]. Available: https://www.kaggle.com/datasets/somesh24/spambase.

[5] UCI Machine Learning Repository – Spambase. [Online]. Available: https://archive.ics.uci.edu/dataset/94/spambase.